

A CANDLE TO SAINT JUDE, OR FOUR REAL WORLD APPLICATIONS OF INTEGER PROGRAMMING¹

R. E. D. WOOLSEY

Institute for Operations Research
Colorado School of Mines

R. E. D. WOOLSEY is principal scientist in The Institute for Operations Research, and professor at the Colorado School of Mines. He is co-owner and operator of the Woolsey Ranch in Williamson County, Texas, and a consultant to Sandia Corporation and the Information Systems Division of General Electric Company in the general areas of operations research and mathematical/integer/geometric programming. He has previously held positions with Control Data Corporation and Sandia Corporation, where he authored several major research reports. His basic interest is the study of methods to solve real world problems.

It has long appeared to this author that a musical analogy to the papers that appear in Management Science and Operations Research is possible. The progression from problem definition to final solution could be considered as a concert by the Philadelphia Orchestra playing Verdi's "Grand March" from *Aida*, followed by the first minute and a half of Strauss' *Also Sprach Zarathustra*.

Unfortunately, those of us who labor in other than academic vineyards discover that if a musical analogy were to be applied to our work in integer programming, a somewhat different concert might be considered: The *William Tell* Overture played by Spike Jones & His City Slickers, followed by the Mormon Tabernacle Choir singing the Bach chorale "Come Soothing Death."

We have, therefore, come to the same point of confusion as Alice where she finds out that what the song is called is one thing, the name of the song is another, and what the song actually is is something else entirely. We, therefore, have the appearance of great progress, some real progress, and damn little real application.

This author has discovered the interesting fact from his experience that on many real world problems, people with experience can beat an elegant theoretical method all the time. If one were to accept this fact as truth, heresy though it may be, one would be driven to the even more embarrassing conclusion that maybe the way to do some things better is to ask the experienced people and then simulate *them*. Of course one must agree that this method flies in the face of all of the theory that says, "Work from the theory to the model to the world." The author has had, however, enough bitter experience with the latter method, to find that it has no useful purpose other than for pelting adversaries at learned society meetings.

¹The author wishes to thank Mr. Allan E. Anglund of the Mountain States Telephone Company for his substantial help in making this manuscript more readable.

The First Candle: Operator Scheduling at a Telephone Company

This problem is something of a classic in the literature found under the name of the "Attendants Rota Problem." More simply stated, the problem is to minimize the cost of having some number of people on duty at work stations in such a way as to "cover" the demand for a certain number of people on duty at any given time. This was first formulated as an integer programming problem.

The problem presented was one with 96 fifteen-minute time bands dividing the day, and 70 possible tours to be worked by the operators. The company had set an upper bound equal to the number of possible positions of 30 operators on any given tour of duty. If this problem were formulated as an integer programming problem, we would have 350 variables with which to contend. This in itself might not be so bad; however, the number of tours could go as high as 500. The company has tried this 96×70 problem on one code, and had realized an optimum solution in a reasonable amount of time. The real problem, however, lay in obtaining answers every day to problems of this size. Horror stories abound about codes that converged on one problem and then did not converge on the next when *one* coefficient was changed *one* unit. The first cut at the problem is named the "Mickey Mouse" Method due to the simple-minded nature of the approach. Given the problem stated above, let us assign a starting value to the upper bound on the number of people that can usefully work on a given tour. For each row, we then calculate a measure of how much slack is available to be removed from that row. After all the slacks are calculated, we then determine how much we can reduce each variable without becoming infeasible.²

It was quickly determined that the Mickey Mouse Method was highly sensitive to the way in which the data was presented to the method. This was because ties were very frequent in the early stages of the computation. Ties were broken by choosing the left most column.

After considerable tinkering, Mickey Mouse was replaced by an implementation of the Senju-Toyoda algorithm [3] which is considerably more sensitive to interrelated reductions. The Senju-Toyoda method was consistently more successful in minimizing the number of operators on duty than was Mickey Mouse. In no case in the author's experience did Mickey Mouse do better than Senju-Toyoda; but there were a respectable number of ties particularly on small problems (up to 6 variables).³

Unfortunately, it was found that the two heuristics above could *sometimes* match what was being done by the telephone company people. It was found that two women from the Traffic Department who were ex-chief operators could lay out and arrange line segments and do consistently better than the two heuristic programs. The Gomory cutting plane code *could* do better, but when confronted with possible exponential increases in running time, this was not seriously considered.

²This method is a variation of the method proposed by Riesco and Thomas [1] in June of 1969 for the solution of some special types of linear programming problems. This method has been found to be extremely effective for getting "good" answers to some of the celebrated fixed charge problems of Haldi. For example see [2].

³Note: The Senju-Toyoda program is in [7].

Needless to say, the study was terminated forthwith, and, therefore, the phone company continued to do better than they would have if they had automated the process.

There are, however, some interesting extensions to the above. A Gilmore-Gomory procedure could be used to generate the "optimum" set of tours such that you might be assured that a "good" minimum number of people could be required. The danger in this idea, however, lies in the possibility that the "optimum" set of tours might not be the best set in terms of real world considerations. This interesting situation might take place because the Gilmore-Gomory procedure allows tours to take on nonintegral values at the "optimum" solution. Also, it should be pointed out that what the union might find acceptable in generated tours and those satisfactory to management could differ to such a degree as to create a problem right here.

In conclusion, it would appear that the best method of approaching such a problem as given above is to have a very intimate understanding of *how* it is being solved *at present*. This means considerable time must be taken to determine *how* the people are choosing the tour assignments. After a solid understanding of the *used* decision rules are defined, the analyst might have a slim chance of bettering what is presently being done *if* time sharing is used; time-sharing, because we can, therefore, use the acquired experience of the people doing it, combined with the ability of the computer to do the "dog" work. The author only hopes that those that study this history will not be moved to repeat the attack outlined above.

The Second Candle: Cutting Stock For Rebars

A large construction firm has a design program that will generate a report listing the number of reinforcing bars (rebars) of various lengths needed for the construction of a given concrete foundation or building or bridge. They wanted a program to follow the above design program that would take the rebar report and generate the optimum cutting pattern to cut the needed rebars from standard length stock. The final result desired was a report that would list exactly how to cut the rebars so as to minimize waste. Old OR hands will recognize that this problem is the old one-dimensional cutting stock problem discussed by Gilmore and Gomory in [4].

The Gilmore-Gomory procedure solves the above problem as a *linear* programming problem, which implies that the number of times a cutting pattern is used may be a fractional value. The true elegance of the Gilmore-Gomory method is that it generates a series of "better" columns (cutting patterns) that will eventually drive the waste to a minimum. The generation of these patterns may be done by various ad hoc methods, or optimally by a dynamic program that selects the "best" pattern to enter the basis at any given step. Eventually, as mentioned before, an optimal set of cutting patterns is generated that will minimize waste, given that fractional values for patterns may appear.

In short, we may discover that in order to get minimum waste, we must use cutting pattern 1 2.74 times, cutting pattern 2 34.67 times, and cutting pattern 3 56.33 times. If we round up the solution to the next integer one might suspect that the result is very close to the true optimum solution. The author is prepared to state that this happy situation will happen only under some rather restricted conditions to be developed more fully in what follows.

The author had constructed a batch-mode code following the instructions in the Gilmore-Gomory paper referred to above. The code had an imbedded dynamic program to calculate the best patterns at each step, and printed as final output the (unrounded) number of times each cutting pattern was to be used. The code was extensively tested on problems ranging in size up to 30 lengths, with what appeared to be good results.

At this point the construction firm submitted a group of bench mark problems to test the code. The shop cutting patterns generated by the firms schedulers were also supplied so that this author could have a yardstick against which to measure his method's improvement. The first group of 10 problems were then run and compared with the shop cutting patterns generated by the two schedulers. These problems ranged from 3 to 8 different lengths.

In no case was the algorithm able to even *match* the results supplied by the schedulers. It was usually found that the optimum cost found by the algorithm (before rounding) was less than that found by the schedulers. In every case, rounding up the answer gave more bars used than the scheduler's answer. A number of more careful rounding algorithms were tried, with the result that a match was found in 2 cases (7 and 8 lengths). In three other cases, the carefully rounded result was off by one bar.

Just out of curiosity, the author then solved 5 of the generated "optimal cutting pattern" problems as integer programming problems. This exercise required three different integer programming codes. It would be natural to expect that if we solve the integer programming problem with an "optimal set" of cutting patterns, we should expect to find the "optimum" solution. In other words, the solution should be optimum in the sense that the number of bars required by the algorithm should be less than or equal to the number required by the schedulers.

It must now be recalled that the algorithm is optimum *only* if we allow fractional numbers of patterns to appear. It, therefore, should *not* be expected that an "optimum" solution will be found even if the "optimum" set of cutting patterns generated by the algorithm are used. In two of the five cases, matching of number of bars was found; but in *no* case were the cutting patterns generated by the algorithm and those generated by the schedulers the same.

At this point the author contacted Dr. Paul Gilmore for clarification of some of the above findings. Dr. Gilmore pointed out that considerable thought must go into the rounding algorithm, and that, in almost every case, the rounding must be done with a very comprehensive understanding of the characteristics of the particular situation. Dr. Gilmore further suggested that if combinations of the problems were run together, one might expect better results. This was indicated because, with more different lengths available, rounding should not be as crucial as with smaller problems.

This suggestion was implemented at once. In every case, combinations of two or more of the original problems did as well or better than each running alone. A definite correlation was noticed as larger and larger combinations were tried with better results. After running a few dozen more cases, the author developed the following rule.

RULE: Run as many different lengths at one time as possible until the time to do the internal dynamic programming starts to make expensive increases in time.

It should also be pointed out at this time that Gilmore and Gomory went on in further papers to discuss ways to optimize the dynamic programming process for such problems in [5] and [6].

Dr. Gilmore also suggested that "dogs" be removed before starting the calculation. A "dog" is any length that is required in a very limited quantity relative to the requirements of the other lengths. This one length generates a cutting pattern that will often use up a least one bar. In cases where "dogs" were removed and then added back in to one of the existing cutting patterns (if possible), improvement was *usually* noted. But still, it was found that the best the algorithm could do on small (up to 8 lengths) problems was to match the results found by the schedulers.

When run on problems of larger (10 and 30) lengths, the algorithm could consistently match or do better than the schedulers, but only with careful removal of "dogs" and intelligent adding back in of the "dogs" removed.

The natural suggestion is to combine the smaller problems into large ones, where the chances are that the algorithm will do better. Unfortunately, this procedure is not possible in the rebar problem for two reasons given by the company. One difficulty is that often each individual order is cut at different sites. The second difficulty is that the cutting shop has no procedures and no room for the necessary housekeeping of the considerable in-process inventory that could result from combining a fair number of different orders.

The company stated that if they had the necessary facilities to take care of the expected level of in-process inventory they would just inventory the waste in the first place, thus eliminating the problem.

Further computational experience indicated, on the basis of the benchmarks supplied, that the program would not consistently do better than the two-shop schedulers could do with no computational aids whatsoever. This author then proposed a program to conversationally schedule the cutting patterns. This program would use the experience of the schedulers combined with either an ad hoc or dynamic programming method of pattern generation.*

The company, however, found this approach unacceptable; as it was the firm's desire to automate this process totally and run the combination of programs in batch mode, and eliminate the two scheduler's positions.

Now the above candle does not disprove in any way the work of Gilmore and Gomory in the development of the cutting stock algorithm, which has been proven to be of considerable benefit to the paper industry, among others. It *does* show, however, that it takes a very knowledgeable person *within the industrial framework* of the problem at hand to design a rounding procedure and "dog" procedure to do the real world problem. The author must publicly admit his shortcomings in this area, and can only hope that others may profit from his example.

In most of the standard texts of MS/OR, integer programming is treated just as any other mathematical programming technique. The fact that most of the algorithms proposed are extremely difficult to use in practice is simply ignored. To be blunt, the difference between theory and practice in integer programming can only be marked by the thousands of dollars spent getting these algorithms to converge in an economic amount of time; because the Management Scientist says that the algorithm will "converge in a finite

*This program is given in [7].

number of steps" is no guarantee that the company might not be bankrupted by the expense first. Thus, many of those who actually solve problems turn to heuristic methods to get good starting solutions, followed by some kind of branch and bound scheme to take every possible advantage of problem structure. But these methods are often given short shrift in OR/MS texts and journals due to their "inelegance." One recalls the remark of Marshall Joffre upon seeing one of the world's last cavalry charges in WWI: "It is magnificent, but is it war?"

The Third Candle: Capital Budgeting For Research & Development

An R & D supervisor discovered, at the end of the first quarter of his fiscal year, that he did not have sufficient resources to continue with all of the ongoing 15 projects. He then required his manager of research and his manager of development to submit a list of what projects were to continue, and what projects were to be terminated. After the customary agonizing reappraisal, 8 projects were selected to be continued and this list was submitted to the R & D supervisor for final approval. The supervisor expressed some reservations about the choices that had been made by his managers, particularly when he discovered that the selection process had been simply to choose the projects with the greatest expected value until the money ran out. The supervisor requested that this author aid these managers in the selection process, and suggested that an integer programming model of the capital budgeting problem be used.

The managers were both asked to rank the projects in order of decreasing importance. After this had been done, they were asked to compare lists and to negotiate any difference of opinion. Then both managers were asked to assign a value of 100 "youdels" of satisfaction to the first project. On the basis of the most important project being assigned 100 youdels, they were then asked to assign youdel values to the other projects. Lively differences of opinion developed. When ever this occurred, they were asked to list the exact causal factors that influenced their opinion. This list was added to as we proceeded through the list of projects. After all the projects had been assigned youdel values, we then summed the youdel values for all 15 projects and divided this number into each individual youdel value. Now, having normalized the youdel values, we can consider that the new values reflect the percentage contribution of each project to total satisfaction.

The managers were then asked to define constraining factors on their actions. Both managers agreed that they were really only constrained by graduate engineer man-hours available and engineering associate man-hours available. The managers were able to supply estimates of man-hour requirements in the two groups for each project, and also the known availability of the hours in each group. We then defined the problem as a Weingartner capital budgeting problem.

When this problem was laid out in matrix form the managers noticed at once that the column of coefficients gave some very useful information about relative merit of projects. The ratio of estimated payoff (youdels) to engineer man-hours gave a measure of how effective the project was in terms of engineer man-hours, as did the ratio of payoff to engineering associate man-hours. A case was noted at once where two projects with considerable different payoff values gave exactly the same payoff/engineering value, the associate man-hours being zero for both projects.

However, it was found that this problem had some other useful characteristics. Five of the 15 projects required associate man-hours and the totals of these were 2,700 man-hours with a pool of 5,600 man-hours available. In short, the second constraint was not binding. We may then reduce the above formulation to a knapsack problem. A fast heuristic may then be applied by ranking the variables from left to right in decreasing order of payoff/engineering man-hour. This may be referred to as the "Bang for Buck" ordering. When projects were selected by choosing them in order of biggest "Bang for Buck," it was discovered that 12 projects could be continued rather than the 8 that had been chosen before.

At this point the manager expressed concern that the second constraint had not contributed to the ratio. He suggested that the ratio payoff/ (engineering and associate man-hours) would be more meaningful. It was then pointed out that this new ratio implied that the associate hours were as important as the graduate engineer hours.

He then suggested that because he paid his engineering associates on the average $2/3$ as much as he paid his engineers, the correct ratio should be payoff/ (engineering and $2/3$ associate). It was again pointed out that if all the projects that required engineering associate hours were done, a surplus of over 50% would still remain. The other manager then pointed out that perhaps the reason that they could not do all the projects was that there were too many engineering associates and not enough engineers. After iterating through the above discussion of the surplus for roughly one and three quarter hours, the first manager said, "I think I see what you guys are saying. If we do *all* the projects, we still have 2,900 hours of associates time left and . . . that's . . . why . . . we have so damn much trouble keeping our E.A.S. busy ! ! !" He then asked if the right answer might be to fire some associates or transfer them and hire some more engineers. The author pointed out that it was not the function of the OR analyst to suggest policy, but only to aid the manager to make effective use of his experience because that was what the hell he, the manager, was paid for.

Further, this author repeated the Axiom of Consulting, which is

If I make the decision, and I am proven right, you will never remember.

If I make the decision, and I am proven wrong, you will never forget!

The actual decision that was made was to fire or transfer 7 associates and hire 4 more engineers, which would allow the continuation of all 15 projects.

Let us carefully note that integer programming was not *actually* used. And the solution arrived at did not require the solution of the *integer programming* problem. What the real problem was, was rather to present a solution method that required only an understanding of the manager's job.

Two things must be realized in the above case study. The first rule of OR is that you must formulate the problem in the jargon used by the manager, *not yours*. The second rule is that managers by and large could care less about the sophistication of your method, they want to solve the problem with data *they* put in, and be able to read off the answer when it comes out, in *their terms*.

The Fourth Candle: Allocation of Sales Districts to Salesmen

A company which does most of its business by telephone had the problem of allocation of some 400 sales districts to 40 salesmen. An expected revenue and number of sales contacts had been calculated for each district.

The problem was to allocate salesmen to district in such a way as to make the expected revenue and number of contacts as nearly equal for each salesman as possible. The company had formulated this problem as an integer programming problem which minimized the variance of the expected revenue and sales contacts for the districts over the salesmen. This problem, as formulated, had 1,600 variables and 440 constraints.

Before proceeding to answer the question put by the company on the feasibility of attempting this problem, we asked what was presently being done. We were told that two "little old ladies" had been doing this problem manually for some years. We then asked if the OR group had ever thought to study them to see what they were doing. We were told that they had not done so. With the light of the first three candles in our eyes, we asked how well those little old ladies did at minimizing variance. Upon being told that the difference among the salesmen was usually around 2%, we delivered ourselves of our professional opinion: "Give those little old ladies a raise!"

REFERENCES

1. A Riesco and M. E. Thomas, "A Heuristic Solution Procedure for Linear Programming Problems with Special Structure," *AIIE Transactions*, June 1969, pp. 157.
2. R. E. D. Woolsey, "On Telling It Like It Is in Integer Programming," *Institute of Operations Research Research Report No. 1*, Colorado School of Mines, June 1970.
3. Shizuo Senju and Yoshiaki Toyoda, "An Approach to Linear Programming with 0-1 Variables," *Management Science Application*, Vol. 15, No. 4 (Dec. 1968), pp. 196-207.
4. P. C. Gilmore and R. E. Gomory, "A Linear Programming Approach to the Cutting Stock Problem," *Operations Research*, Vol. 9, No. 6 (Nov.-Dec. 1961), pp. 849-859.
5. P. C. Gilmore and R. E. Gomory, "A Linear Programming Approach to the Cutting Stock Problem - Part II," *Operations Research*, Vol. 11, No. 6 (Nov.-Dec. 1963), pp. 863-888.
6. P. C. Gilmore and R. E. Gomory, "The Theory and Computation of Knapsack Functions," *Operations Research*, Vol. 14, No. 6 (Nov.-Dec. 1966), pp. 1045-1074.
7. R. E. D. Woolsey, *The Operations Research Quick & Dirty Manual*, Harper and Row (to appear).

Copyright 1972, by INFORMS, all rights reserved. Copyright of Interfaces is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.